IPL AUCTION PRO

Product Requirements Document + Master Build Prompt

Version 1.0 | May 2026 | Confidential

FastAPI Backend	Next.js Frontend	Claude AI Analysis	Real-time WebSocket

PART 1 -- MASTER BUILD PROMPT

Copy the entire section below and paste into Claude, Cursor, or any AI coding assistant. It contains all specs needed to generate the complete IPL Auction Pro codebase.

--- START OF PROMPT (COPY FROM HERE) ---

PROMPT BOX

You are a senior full-stack engineer and UI/UX designer. Build a complete, production-ready IPL Cricket Auction Simulator called IPL Auction Pro with the complete specifications below.

TECH STACK (MANDATORY):
Backend: Python 3.11 + FastAPI, PostgreSQL (SQLAlchemy async), Redis (room state + timers + pubsub), WebSockets via FastAPI, Anthropic Claude API (claude-sonnet-4-20250514) for AI analysis, Alembic migrations, Pydantic v2.
Frontend: Next.js 14 App Router + TypeScript, Tailwind CSS with custom dark IPL theme, Framer Motion animations, Recharts for analysis charts, Socket.io-client for WebSocket, Shadcn/UI base components, Google Fonts: Clash Display + Satoshi.
Deploy: Backend on Railway, Frontend on Vercel.

DESIGN SYSTEM:
Background Primary: #0A0F1E (deep navy). Background Secondary: #111827. Accent Gold: #F59E0B. Accent Blue: #3B82F6. Text Primary: #F9FAFB. Text Secondary: #9CA3AF.
Typography: Clash Display 700 for headings, Satoshi 400/500 for body, JetBrains Mono for stats.
Cards: bg-[#111827] border border-[#1F2937] rounded-2xl. Primary button: gold gradient. Countdown ring: SVG circular with gold stroke.

FEATURES TO BUILD:

1. ROOM SYSTEM
- Create room: host picks name, IPL team, auction mode, timer setting
- 6-char uppercase alphanumeric room codes
- Lobby: all joined users, team assignments, ready status
- Room state in Redis with 6-hour TTL
- Max 10 bidders + unlimited spectators
- No login required: anonymous sessions with localStorage token

2. AUCTION MODES
a) IPL 2026 Mock Auction: 350+ official players, 42 sets by tier (Marquee/A/B/C), real team retentions, accurate purse deductions
b) Mega Auction: All 230+ top players, no retentions, Rs.120 Cr each team
c) Custom Auction: host creates custom player list, custom budgets, custom bid increments

3. BIDDING ENGINE
- Default countdown timer: 15 seconds (host can set 10/15/20/30s)
- Timer resets by 7 seconds on each new bid
- Bid increments follow real IPL 2026 rules: Rs.20L to Rs.1Cr = +Rs.5L, Rs.1Cr to Rs.3Cr = +Rs.10L, Rs.3Cr to Rs.5Cr = +Rs.20L, Rs.5Cr to Rs.10Cr = +Rs.25L, above Rs.10Cr = +Rs.50L
- Anti-spam: 0.5s cooldown between bids per user
- Budget enforcement server-side: cannot bid beyond remaining purse
- Overseas cap enforcement: max 8 per team
- Bid buttons: +Min Increment, +1Cr, +5Cr, All-In

4. PLAYER DATABASE (350+ PLAYERS)
Each player: id, name, nationality, dob, role (BAT/BOWL/AR/WK), base_price_lakhs, tier,

career_stats: { matches, innings, runs, avg, sr, fifties, hundreds, wickets, economy, best_bowling, catches, stumpings },
recent_form: { last_season_runs, last_season_wickets, last_season_avg },
ipl_history: [{ year, team, price_paid_cr, matches, runs, wickets }],
is_overseas: bool

Include all major players: Virat Kohli, Rohit Sharma, MS Dhoni, Jasprit Bumrah, KL Rahul, Shreyas Iyer, Hardik Pandya, Suryakumar Yadav, Rishabh Pant, Jos Buttler, Pat Cummins, Rashid Khan, Glenn Maxwell, David Warner, Quinton de Kock, Faf du Plessis, Mitchell Starc, Jofra Archer, and all others from IPL 2026 pool across tiers.

5. LIVE AUCTION UI
- Current player card: large display with avatar (initials), role badge, nationality, base price, key career stat
- Animated bid amount display
- Current highest bidder team badge
- SVG countdown ring (gold stroke, circular progress)
- All teams sidebar: budget remaining + squad count
- Recent activity feed: last 5 bids (team + amount)
- Player click: expand modal with full career stats + IPL history

6. POST-AUCTION AI EVALUATION (KEY FEATURE)
After auction ends, host clicks Analyze Teams.
FastAPI sends all squad data + full career stats to Claude API.

Claude API system prompt: You are an expert IPL cricket analyst with 15 years of experience. Analyze team compositions with deep knowledge of player career statistics. Always respond with valid JSON only.

Claude must return this JSON structure:
```
{
  team_rankings: [{ rank, team_name, overall_score, grade }],
  team_reports: [{
    team_name, overall_score (0-100), batting_depth (0-100), bowling_balance (0-100),
    all_round_strength (0-100), value_for_money (0-100), overseas_balance (0-100),
    strengths: [3 strings], weaknesses: [3 strings],
    best_xi: [11 player names], captain_pick: string, verdict: string,
    head_to_head: [{ vs_team, win_probability_percent, reason }]
  }],
  auction_highlights: {
    biggest_buy: { player, team, price_cr },
    best_value_pick: { player, team, price_cr, reason },
    worst_value_pick: { player, team, price_cr, reason },
    biggest_rivalry: { team1, team2, reason }
  }
}
```

Display analysis as: animated team ranking leaderboard, radar charts per team (Recharts, 5 axes), head-to-head matchup cards with win %, auction highlights section, Download PDF button.

7. SPECTATOR MODE
- Join without team/bidding rights
- See live bids, squads, commentary
- Emoji reactions: fire, money-bag, shocked (live count broadcast via WS)

8. WEBSOCKET EVENTS
Client to Server: { type: bid, amount }, { type: host_action, action: start/pause/skip/unsold }, { type: reaction, emoji: fire/money/shocked }

Server to Client: player_up, bid_update, timer_tick, player_sold, player_unsold, auction_complete, reaction_update

9. BACKEND FILE STRUCTURE

```
backend/app/main.py, config.py, database.py, redis_client.py
backend/app/models/: room.py, player.py, squad.py, bid_log.py
backend/app/schemas/: room.py, player.py, bid.py, analysis.py
backend/app/routers/: rooms.py, auction.py, players.py, analysis.py
backend/app/websocket/: manager.py, handlers.py
backend/app/services/: auction_engine.py, ai_service.py, player_service.py
backend/app/data/players_seed.py
```

10. FRONTEND FILE STRUCTURE

```
frontend/app/: layout.tsx, page.tsx (home), room/[id]/page.tsx (auction),
room/[id]/analysis/page.tsx, players/page.tsx
frontend/components/auction/: PlayerCard.tsx, CountdownRing.tsx, BidButtons.tsx,
ActivityFeed.tsx, TeamsSidebar.tsx
frontend/components/team/: BudgetTracker.tsx, SquadGrid.tsx
frontend/components/analysis/: TeamRadarChart.tsx, TeamRankCard.tsx,
HeadToHead.tsx, AuctionHighlights.tsx
frontend/lib/: socket.ts, api.ts, types.ts
```

11. REST API ENDPOINTS

```
POST /api/rooms/create -> { room_code, host_token }
POST /api/rooms/{code}/join -> { player_token }
GET  /api/rooms/{code}/state -> RoomState
WS   /api/ws/{code}/{token} -> Event stream
POST /api/rooms/{code}/start
POST /api/rooms/{code}/bid
POST /api/rooms/{code}/host/{action}
GET  /api/players?role=&tier=&nationality=&search=
GET  /api/players/{id}
POST /api/rooms/{code}/analyze
GET  /api/rooms/{code}/analysis
```

12. ENVIRONMENT VARIABLES

Backend: DATABASE_URL, REDIS_URL, ANTHROPIC_API_KEY, SECRET_KEY, CORS_ORIGINS
Frontend: NEXT_PUBLIC_API_URL, NEXT_PUBLIC_WS_URL

ADDITIONAL REQUIREMENTS:

- Money: store in Lakhs internally, display as Cr/L format (e.g., Rs.1.5 Cr)
- Rooms auto-expire 6 hours after last activity (Redis TTL)
- Confetti animation on player sold (canvas-confetti library)
- Toast notifications: outbid alert, budget warning below Rs.20Cr, bid placed confirmation
- Loading skeletons for all async content
- Error boundaries with retry buttons
- Dark mode only (no light mode toggle)
- Mobile-first responsive layout
- Provide complete working code for EVERY file. Start from backend setup, then player seed, then WebSocket auction engine, then frontend pages, then AI integration.

> You are a senior full-stack engineer and UI/UX designer. Build a complete, production-ready IPL Cricket Auction Simulator called IPL Auction Pro with the complete specifications below.

> TECH STACK (MANDATORY):
> Backend: Python 3.11 + FastAPI, PostgreSQL (SQLAlchemy async), Redis (room state + timers + pubsub), WebSockets via FastAPI, Anthropic Claude API (claude-sonnet-4-20250514) for AI analysis, Alembic migrations, Pydantic v2.

Frontend: Next.js 14 App Router + TypeScript, Tailwind CSS with custom dark IPL theme, Framer Motion animations, Recharts for analysis charts, Socket.io-client for WebSocket, Shadcn/UI base components, Google Fonts: Clash Display + Satoshi.
Deploy: Backend on Railway, Frontend on Vercel.

DESIGN SYSTEM:
Background Primary: #0A0F1E (deep navy). Background Secondary: #111827. Accent Gold: #F59E0B. Accent Blue: #3B82F6. Text Primary: #F9FAFB. Text Secondary: #9CA3AF.
Typography: Clash Display 700 for headings, Satoshi 400/500 for body, JetBrains Mono for stats.
Cards: bg-[#111827] border border-[#1F2937] rounded-2xl. Primary button: gold gradient. Countdown ring: SVG circular with gold stroke.

FEATURES TO BUILD:

1. ROOM SYSTEM
- Create room: host picks name, IPL team, auction mode, timer setting
- 6-char uppercase alphanumeric room codes
- Lobby: all joined users, team assignments, ready status
- Room state in Redis with 6-hour TTL
- Max 10 bidders + unlimited spectators
- No login required: anonymous sessions with localStorage token

2. AUCTION MODES
a) IPL 2026 Mock Auction: 350+ official players, 42 sets by tier (Marquee/A/B/C), real team retentions, accurate purse deductions
b) Mega Auction: All 230+ top players, no retentions, Rs.120 Cr each team
c) Custom Auction: host creates custom player list, custom budgets, custom bid increments

3. BIDDING ENGINE
- Default countdown timer: 15 seconds (host can set 10/15/20/30s)
- Timer resets by 7 seconds on each new bid
- Bid increments follow real IPL 2026 rules: Rs.20L to Rs.1Cr = +Rs.5L, Rs.1Cr to Rs.3Cr = +Rs.10L, Rs.3Cr to Rs.5Cr = +Rs.20L, Rs.5Cr to Rs.10Cr = +Rs.25L, above Rs.10Cr = +Rs.50L
- Anti-spam: 0.5s cooldown between bids per user
- Budget enforcement server-side: cannot bid beyond remaining purse
- Overseas cap enforcement: max 8 per team
- Bid buttons: +Min Increment, +1Cr, +5Cr, All-In

4. PLAYER DATABASE (350+ PLAYERS)
Each player: id, name, nationality, dob, role (BAT/BOWL/AR/WK), base_price_lakhs, tier,
career_stats: { matches, innings, runs, avg, sr, fifties, hundreds, wickets, economy, best_bowling, catches, stumpings },
recent_form: { last_season_runs, last_season_wickets, last_season_avg },
ipl_history: [{ year, team, price_paid_cr, matches, runs, wickets }],
is_overseas: bool

Include all major players: Virat Kohli, Rohit Sharma, MS Dhoni, Jasprit Bumrah, KL Rahul, Shreyas Iyer, Hardik Pandya, Suryakumar Yadav, Rishabh Pant, Jos Buttler, Pat Cummins, Rashid Khan, Glenn Maxwell, David Warner, Quinton de Kock, Faf du Plessis, Mitchell Starc, Jofra Archer, and all others from IPL 2026 pool across tiers.

5. LIVE AUCTION UI
- Current player card: large display with avatar (initials), role badge, nationality, base price, key career stat
- Animated bid amount display
- Current highest bidder team badge
- SVG countdown ring (gold stroke, circular progress)
- All teams sidebar: budget remaining + squad count

- Recent activity feed: last 5 bids (team + amount)
- Player click: expand modal with full career stats + IPL history

6. POST-AUCTION AI EVALUATION (KEY FEATURE)
After auction ends, host clicks Analyze Teams.
FastAPI sends all squad data + full career stats to Claude API.

Claude API system prompt: You are an expert IPL cricket analyst with 15 years of experience. Analyze team compositions with deep knowledge of player career statistics. Always respond with valid JSON only.

Claude must return this JSON structure:

```
{
  team_rankings: [{ rank, team_name, overall_score, grade }],
  team_reports: [{
    team_name, overall_score (0-100), batting_depth (0-100), bowling_balance (0-100),
    all_round_strength (0-100), value_for_money (0-100), overseas_balance (0-100),
    strengths: [3 strings], weaknesses: [3 strings],
    best_xi: [11 player names], captain_pick: string, verdict: string,
    head_to_head: [{ vs_team, win_probability_percent, reason }]
    }],
  auction_highlights: {
    biggest_buy: { player, team, price_cr },
    best_value_pick: { player, team, price_cr, reason },
    worst_value_pick: { player, team, price_cr, reason },
    biggest_rivalry: { team1, team2, reason }
    }
}
```

Display analysis as: animated team ranking leaderboard, radar charts per team (Recharts, 5 axes), head-to-head matchup cards with win %, auction highlights section, Download PDF button.

7. SPECTATOR MODE
- Join without team/bidding rights
- See live bids, squads, commentary
- Emoji reactions: fire, money-bag, shocked (live count broadcast via WS)

8. WEBSOCKET EVENTS
Client to Server: { type: bid, amount }, { type: host_action, action: start/pause/skip/unsold }, { type: reaction, emoji: fire/money/shocked }
Server to Client: player_up, bid_update, timer_tick, player_sold, player_unsold, auction_complete, reaction_update

9. BACKEND FILE STRUCTURE

```
backend/app/main.py, config.py, database.py, redis_client.py
backend/app/models/: room.py, player.py, squad.py, bid_log.py
backend/app/schemas/: room.py, player.py, bid.py, analysis.py
backend/app/routers/: rooms.py, auction.py, players.py, analysis.py
backend/app/websocket/: manager.py, handlers.py
backend/app/services/: auction_engine.py, ai_service.py, player_service.py
backend/app/data/players_seed.py
```

10. FRONTEND FILE STRUCTURE

```
frontend/app/: layout.tsx, page.tsx (home), room/[id]/page.tsx (auction),
room/[id]/analysis/page.tsx, players/page.tsx
frontend/components/auction/: PlayerCard.tsx, CountdownRing.tsx, BidButtons.tsx,
ActivityFeed.tsx, TeamsSidebar.tsx
frontend/components/team/: BudgetTracker.tsx, SquadGrid.tsx
frontend/components/analysis/: TeamRadarChart.tsx, TeamRankCard.tsx, HeadToHead.tsx,
AuctionHighlights.tsx
```

```
frontend/lib/: socket.ts, api.ts, types.ts
```

11. REST API ENDPOINTS

```
POST /api/rooms/create -> { room_code, host_token }
POST /api/rooms/{code}/join -> { player_token }
GET  /api/rooms/{code}/state -> RoomState
WS   /api/ws/{code}/{token} -> Event stream
POST /api/rooms/{code}/start
POST /api/rooms/{code}/bid
POST /api/rooms/{code}/host/{action}
GET  /api/players?role=&tier=&nationality=&search=
GET  /api/players/{id}
POST /api/rooms/{code}/analyze
GET  /api/rooms/{code}/analysis
```

12. ENVIRONMENT VARIABLES
Backend: DATABASE_URL, REDIS_URL, ANTHROPIC_API_KEY, SECRET_KEY, CORS_ORIGINS
Frontend: NEXT_PUBLIC_API_URL, NEXT_PUBLIC_WS_URL

ADDITIONAL REQUIREMENTS:
- Money: store in Lakhs internally, display as Cr/L format (e.g., Rs.1.5 Cr)
- Rooms auto-expire 6 hours after last activity (Redis TTL)
- Confetti animation on player sold (canvas-confetti library)
- Toast notifications: outbid alert, budget warning below Rs.20Cr, bid placed confirmation
- Loading skeletons for all async content
- Error boundaries with retry buttons
- Dark mode only (no light mode toggle)
- Mobile-first responsive layout
- Provide complete working code for EVERY file. Start from backend setup, then player seed, then WebSocket auction engine, then frontend pages, then AI integration.

--- END OF PROMPT ---

PART 2 -- PRODUCT REQUIREMENTS DOCUMENT (PRD)

1. Product Overview

Product Name	IPL Auction Pro
Tagline	The smartest IPL auction simulator -- with Claude AI team evaluation
Version	1.0 MVP
Target Users	Cricket fans, IPL enthusiasts, friend groups (4-10 players)
Key Differentiator	Post-auction Claude AI analysis: full career data-driven team report with rankings, radar charts, and head-to-head predictions

1.1 Problem Statement

playauctiongame.com is the closest existing product. It has 3 critical gaps:

- No AI evaluation -- players have no way to know if their team is good after the auction ends
- No career/lifetime stats for players -- bidding is done without data
- Basic UI -- no premium experience, no animations, no depth

1.2 Vision

A free, no-signup IPL auction simulator where 4-10 friends bid on real IPL players in real-time, with a premium dark IPL-grade UI, full player career stats, and a post-auction Claude AI report that ranks all teams and predicts match outcomes based on lifetime player data.

2. Competitive Analysis

Feature	playauctiongame.com	IPL Auction Pro
Real-time multiplayer bidding	Yes (basic)	Yes (WebSocket + Redis)
No signup required	Yes	Yes
Player career stats	None	Full IPL career data
AI team evaluation	None	Claude API full analysis
Post-auction team rankings	None	Animated leaderboard
Head-to-head win predictions	None	Win probability %
Bid advisor (fair value)	None	AI-suggested price
Spectator mode + reactions	Limited	Full spectator + emoji

Custom auction mode	None	Custom player list
Mobile responsive	Basic	Mobile-first premium
Radar chart analysis	None	5-axis per team
Design quality	Basic dark	Premium IPL-grade dark UI

3. User Stories

3.1 Core User Stories

1. As a host, I want to create a room and share a 6-char code so my friends can join instantly without signing up.
2. As a bidder, I want to see player career stats before bidding so I can make informed decisions.
3. As a bidder, I want real-time bid updates so I know when I have been outbid instantly.
4. As a bidder, I want to track my remaining budget and squad composition during the auction.
5. As a player, I want to see a countdown timer that resets on each bid so the auction feels live.
6. As a host, I want to control auction pace: start, pause, skip player, mark unsold.
7. As all players, I want an AI report after the auction that ranks our teams and explains why.
8. As a spectator, I want to watch the auction live and react with emojis.

4. Functional Requirements

4.1 Room Management

- Room creation: name, IPL team selection, auction mode, timer setting
- 6-character alphanumeric room codes (uppercase, e.g., IPL2K4)
- Lobby shows all joined users, team assignments, ready status
- Room state persisted in Redis with 6-hour TTL
- Maximum 10 bidding participants + unlimited spectators

4.2 Bidding Engine

- Players presented one-by-one in predefined set order
- Configurable countdown timer: 10s / 15s / 20s / 30s (default 15s)
- Timer extends by 7 seconds on each new bid
- Bid increments follow real IPL 2026 official rules
- Auto-advance to next player when timer hits zero (unsold)
- Budget enforcement: server-side, cannot overbid remaining purse
- Overseas cap: max 8 per team, enforced server-side

4.3 AI Analysis Engine

- Triggered by host after auction completes
- Collects full squad data + career stats from PostgreSQL

- Sends structured prompt to Claude API with all team + player data
- Returns team scores (0-100) across 6 dimensions per team
- Returns strengths, weaknesses, best XI, captain pick per team
- Returns head-to-head win probabilities for all team pairs
- Returns auction highlights: best buy, worst buy, biggest rivalry
- Results cached in DB -- shareable link to analysis page

5. Data Models

5.1 Room

Field	Type	Description	Example
id	UUID	Primary key	uuid4
code	str	6-char room code	IPL2K4
host_token	str	Host auth token	tok_xxx
mode	enum	Mock/Mega/Custom	mock_2026
status	enum	lobby/active/complete	active
timer_seconds	int	Default bid timer	15

5.2 Player

Field	Type	Description	Example
name	str	Full player name	Virat Kohli
role	enum	BAT/BOWL/AR/WK	BAT
nationality	str	Indian/Overseas	Indian
base_price	int	Base price (Lakhs)	200
tier	enum	Marquee/A/B/C	Marquee
ipl_matches	int	Career IPL matches	237
ipl_runs	int	Career IPL runs	7263
ipl_avg	float	Career batting avg	36.20
ipl_sr	float	Career strike rate	130.41
ipl_wickets	int	Career wickets	4
ipl_economy	float	Career economy	8.82

6. FastAPI REST Endpoints

Method	Endpoint	Description	Response
POST	/api/rooms/create	Create new auction room	{ room_code, host_token }
POST	/api/rooms/{code}/join	Join room as bidder	{ player_token, team }
GET	/api/rooms/{code}/state	Get current room state	RoomState object
WS	/api/ws/{code}/{token}	WebSocket connection	Event stream

POST	/api/rooms/{code}/start	Host starts auction	{ success: true }
POST	/api/rooms/{code}/bid	Place a bid	{ new_amount, leader }
POST	/api/rooms/{code}/host/{action}	Host control action	{ success: true }
GET	/api/players	Get players with filters	Player[] paginated
GET	/api/players/{id}	Get player detail	PlayerDetail
POST	/api/rooms/{code}/analyze	Trigger AI analysis	{ job_id }
GET	/api/rooms/{code}/analysis	Get analysis results	AnalysisReport

7. Non-Functional Requirements

WS Bid Latency	Below 100ms broadcast to all room members
Concurrent Rooms	50+ simultaneous rooms (Railway free tier)
AI Analysis Time	Below 15 seconds via Claude API
Page Load (LCP)	Below 2 seconds on mobile 4G
Session Security	JWT tokens in localStorage, room-scoped permissions
Data Expiry	Rooms auto-expire after 6 hours (Redis TTL)

8. Build Phases

Phase 1 -- Backend Foundation (Week 1)

9. FastAPI project setup with PostgreSQL + Redis + Alembic migrations
10. Player model + seed 350+ players with accurate career stats
11. Room CRUD API: create, join, get state
12. WebSocket ConnectionManager: connect, disconnect, broadcast
13. Bid validation engine with timer + increment logic

Phase 2 -- Frontend Core (Week 2)

14. Next.js 14 setup with Tailwind dark IPL theme + Clash Display + Satoshi fonts
15. Home page: create room / join room forms
16. Lobby page: team list, ready toggle
17. Auction room: PlayerCard, CountdownRing, BidButtons, TeamsSidebar
18. WebSocket client integration + real-time bid updates

Phase 3 -- AI Integration (Week 3)

19. Claude API integration in ai_service.py
20. Analysis endpoint: collect squads, format prompt, parse JSON response
21. Analysis page: radar charts (Recharts), team rankings, head-to-head
22. Bid Advisor: show AI fair value per player in auction
23. Player profile modal with full career stats + IPL history

Phase 4 -- Polish + Deploy (Week 4)

24. Confetti animation on player sold (canvas-confetti)
25. Spectator mode + emoji reactions via WebSocket
26. Mobile responsive audit + fixes
27. Loading skeletons + error boundaries + toast notifications
28. Deploy backend to Railway, frontend to Vercel

9. Technology Stack Summary

Layer	Technology	Purpose
Backend Framework	FastAPI (Python 3.11)	REST + WebSocket server
Primary Database	PostgreSQL + SQLAlchemy	Rooms, players, squads, bids
Cache + PubSub	Redis	Room state + timers + broadcast
AI Engine	Anthropic Claude API	Team analysis + bid advisor
Frontend Framework	Next.js 14 App Router	React SSR + routing
Styling	Tailwind CSS	Dark IPL theme
Animations	Framer Motion	Cards, transitions
Charts	Recharts	Radar charts + budget bars
Type Safety	TypeScript	Full-stack
Backend Deploy	Railway	Free tier + auto-deploy
Frontend Deploy	Vercel	Free tier + edge CDN